ATELIER 7 Le rapport de 6h

Algorithmie comparative - J3 - Mardi 10 novembre 2026, 15h05 - 120 minutes, en binôme

La question à laquelle l'atelier répond

Le premier jour ouvré du mois, à 6h, le rapport des cent premiers clients fait tomber le serveur : mémoire saturée. Comment sortir les cent premiers sans ranger les trois millions de lignes ?

Ce que vous produisez : adr/GRILLE-ADR-004.md et adr/ADR-004-rapport.md (carnet), un CSV et une courbe dans bench/resultats.

Le rendu attendu

En fin d'atelier : le code, les mesures et la grille passent le correcteur, et l'ADR existe en premier jet (chaque rubrique en une à trois lignes, avec vos chiffres et le nom de vos CSV). Le correcteur sans KO avant 17h00, le commit du binôme à 17h00. Ensuite, l'ADR s'améliore chez vous jusqu'au dépôt du carnet, le 1er décembre.

Avant 17h00, le correcteur de l'atelier répond sans KO :

```
OK  Le mois de facturation est généré : trois millions de lignes
OK  À COMPLÉTER : le tas de taille K donne les cent mêmes clients que le legacy
OK  Les trois variantes sont mesurées jusqu'à trois millions de lignes, mémoire comprise
OK  La courbe est tracée
OK  La grille comparative de l'ADR-004 est remplie
OK  L'ADR-004 a ses six rubriques, remplies
OK  La rubrique 3 de l'ADR-004 renvoie à une mesure qui existe, la rubrique 5 donne le domaine de validité
OK  Les trois phrases de l'ADR-004 tiennent sans jargon
8 contrôles, 0 KO
```

Avant de commencer. Votre tas de l'atelier 6 passe le correcteur 6 : le tas de taille K s'appuie sur lui. Sinon, téléchargez correction_tas.zip sur Moodle et décompressez-le à la racine du dépôt :

Windows	`Expand-Archive -Path $HOME\Downloads\correction_tas.zip -DestinationPath . -Force`
macOS, Linux	`unzip -o ~/Downloads/correction_tas.zip`

Copiez la grille. Écrivez-y tout de suite les poids et les cinq barèmes du sous-objectif 5 :

Windows	`Copy-Item adr\grille-comparative.md adr\GRILLE-ADR-004.md`
macOS, Linux	`cp adr/grille-comparative.md adr/GRILLE-ADR-004.md`

Aujourd'hui, la fiche atelier en cours donne la première commande de chaque sous-objectif et ce que vous devez voir. Les autres commandes sont celles des fiches atelier numéro 2 à 4, et toutes sont dans le README du dépôt. Les consignes de la slide sont rappelées en face de chaque sous-objectif.

Les sous-objectifs

1. Générer le mois de facturation consigne 1 de la slide

Pourquoi : Une ligne par prestation facturée en octobre 2026, à graine fixe : trois millions de lignes, 1 900 clients, 110 Mo environ.

Faites : Lancez la génération (quelques secondes) ; pendant ce temps, lisez les lignes 32, 39 et 40 de legacy/src/rapport.ts.

Les trois systèmes	`npm run mois`

Vous devez voir : Le même nombre de lignes, de clients et d'euros sur tous les postes.

```
jeu mois : 3000000 lignes de facturation, 1900 clients, 1 071 429 euros facturés
```

2. Compléter le tas de taille K, lancer les tests consigne 2 de la slide

Pourquoi : Le sommet du tas est le moins bon des K gardés : c'est lui qu'on compare au client suivant.

Faites : Complétez l'unique À COMPLÉTER de topKParTas, dans bench/variantes/rapport.ts, puis lancez les tests.

Les trois systèmes	`npm test`

Vous devez voir : Les trois tests du rapport passent : les cent mêmes clients que le legacy, dans le même ordre. Si vos ateliers 5 et 6 sont faits, npm test ne signale plus aucun échec.

```
✔ les trois variantes du rapport donnent le même classement (20.4013ms)
✔ à total égal, le plus petit numéro de client passe devant (0.6388ms)
✔ moins de clients que K : tous, dans l'ordre (0.4474ms)
```

3. Mesurer le temps et la mémoire consigne 3 de la slide

Pourquoi : Trois variantes, de 100 000 à trois millions de lignes ; npm run bench force le garbage collector avant chaque appel.

Faites : Lancez la mesure (une à deux minutes), toujours par npm run bench.

Les trois systèmes	`npm run bench -- --sujet rapport --repetitions 5 --echauffement 2`

Vous devez voir : Vos durées seront différentes, l'ordre de grandeur de la mémoire non.

```
variante        taille   duree_ms   memoire_ko
tri            3000000    1099.08        66516
tas            3000000      64.39          297
quickselect    3000000      86.25        19459
```

Même avec npm run bench, memoire_ko peut sauter d'une répétition ou d'un lancement à l'autre : le garbage collector passe ou non. Lisez la mémoire à 3 millions de lignes. Si un relevé paraît absurde, relancez une fois, gardez la plus petite valeur positive des deux et dites-le en rubrique 3.

4. Lire la mémoire et la courbe consigne 4 de la slide

Pourquoi : Le ticket a deux contraintes, la mémoire de pointe et le temps : la mesure dit de combien chaque variante les tient.

Faites : Comparez le temps et la mémoire des trois variantes à trois millions de lignes, puis tracez la courbe. La mémoire se lit dans le tableau final de la mesure ; la courbe ne trace que le temps.

Les trois systèmes	`npm run courbe -- --log`

Vous devez voir : Les médianes de temps par variante, puis le fichier SVG à ouvrir dans un navigateur.

```
courbe écrite : bench\resultats\2026-11-08T17-31-08-rapport.svg
```

> **À préparer pour l'ADR :** À trois millions de lignes, combien de fois le tri dépasse-t-il le tas en mémoire, et en temps ? Lequel des deux écarts répond au ticket ?

5. Remplir la grille, écrire l'ADR-004 consignes 5 et 6 de la slide

Pourquoi : Les poids du ticket : temps 2, mémoire 4, lisibilité 2, robustesse 1, coût 1. Mémoire, à trois millions de lignes : 5 sous 10 Mo, 4 sous 50, 3 sous 200, 2 sous 1 Go, 1 au-delà ; memoire_ko est un ordre de grandeur, et deux relevés à moins d'un facteur 2 l'un de l'autre prennent la même note. Temps : 5 sous 0,5 s, 4 sous 2 s, 3 sous 10 s, 2 sous 60 s, 1 au-delà. Lisibilité : partez de 5 et retirez un point par idée qu'il faut connaître pour lire la variante (le tas et son sommet qui est le moins bon ; le pivot, la descente dans une seule moitié et le tirage à graine fixe). Robustesse : 5 si le classement est exact sur les trois tests avec un pire cas garanti ; 3 si le classement est exact mais le pire cas quadratique, rendu improbable par un pivot tiré au hasard (la note 4 n'est pas utilisée) ; 2 si un cas courant (moins de clients que K, deux totaux égaux) donne un classement faux en silence ; 1 si npm test échoue. Coût : 5 si rien ne change ; 4 pour quelques lignes et une structure déjà écrite ou fournie ; 3 pour une structure à écrire et à tester ; 2 pour une donnée à préparer à chaque lancement ; 1 pour une dépendance ajoutée.

Faites : Remplissez la grille. Copiez le gabarit d'ADR, puis écrivez l'ADR-004. Préparez la décision et la question de compréhension à deux, puis lancez `npm run verifier -- 7`.

Windows	`Copy-Item adr\ADR-000-gabarit.md adr\ADR-004-rapport.md`
macOS, Linux	`cp adr/ADR-000-gabarit.md adr/ADR-004-rapport.md`

- Vous avez 120 minutes : écrivez l'ADR-004 et sa grille en entier. Le temps qui reste sert à réparer les KO des correcteurs 5 et 6, puis à compléter vos ADR-002 et ADR-003.
- Rubrique 3 : un tableau, le temps et la mémoire à 100 000, 1 000 000 et 3 000 000 de lignes, à la place des colonnes du gabarit ; votre CSV.
- Rubrique 5 : le chargement des lignes, non mesuré ; le nombre de clients à partir duquel la Map des totaux passerait 10 Mo (la note 5 de mémoire) : divisez memoire_ko du tas à 3 millions de lignes par les 1 900 clients.

Vous devez voir : Avant 17h00, le correcteur sans KO, comme en première page : 8 contrôles, 0 KO, le tas de ce matin compris.

> **La décision de l'atelier**
> Quelle variante retenez-vous pour le rapport ? Si deux variantes se valent sur la mesure, qu'est-ce qui les départage ? C'est la rubrique 4 de l'ADR-004, en une phrase.

> **La question de compréhension**
> À quoi sert ce que vous venez de faire, en deux phrases ? Préparez-la à deux : l'intervenant passe la demander à chaque binôme pendant l'atelier.

Au besoin

Où trouver un exemple dans le code

bench/variantes/rapport.ts contient les trois variantes : totauxParClient calcule les totaux en une passe avec une Map, quickselect est écrite en entier. Le tas de l'atelier 6 reçoit sa comparaison à la construction : meilleur(b, a) met le moins bon au sommet.

Si ça coince

- Les tests du rapport échouent sur l'ordre : la comparaison est inversée, le meilleur est au sommet du tas.
- Le harness s'arrête sur ARRÊT avec le sujet rapport alors que vos tests passent : le tas de ce matin est faux ; le correcteur 6 vous le dit, correction_tas.zip est sur Moodle.
- memoire_ko négative : le harness a été lancé sans npm run bench.

Toujours bloqué après ces trois pistes : levez la main, ou écrivez sur Teams.

Si vous avez fini

- Écrivez la requête SQL qui ferait le même rapport dans la base, puis ajoutez sous la rubrique 5 ce qu'elle supprimerait.
- Relancez le tas et le quickselect avec neuf répétitions, puis écrivez sous la rubrique 3 si leur écart de temps dépasse le bruit de la mesure :

  Les trois systèmes

  ```
  npm run bench -- --sujet rapport --variantes tas,quickselect --tailles 3000000 --repetitions 9 --echauffement 2
  ```

Annexe : les cas particuliers

Le harness se lance toujours par npm run bench. Appelé directement (`npx tsx bench/harness.ts`), il ne force pas le garbage collector : memoire_ko devient fausse, parfois négative.

Le mois de facturation ne se versionne pas : le .gitignore l'écarte. S'il manque de place sur le disque, les jeux de data/jeux se régénèrent.

Les sorties montrées sont de vraies sorties, capturées en jouant la fiche atelier en cours ; vos durées seront différentes, vos nombres de lignes, de clients et vos sommes de contrôle non. Avant 17h00, le binôme commite : git add -A puis git commit -m "J3". Sans commit, le travail du jour n'est pas évalué.